DATABASE MANAGEMENT SYSTEMS LABORATORY

DBMS Lab Experiments Manual

Complete Documentation & Query Execution Reports

Student Name: Rishabh Tiwari

Experiments Included: Experiment 03, 04, 05, and 06

Database: MySQL / CompanyDB

EXPERIMENT - 3

AIM

To create an Employee–Department–Project relational database, insert at least 30 employees across 5 departments and 8 projects, and demonstrate SQL queries using Selection, Projection, Aggregate Functions, GROUP BY, HAVING, CASE expressions, and ORDER BY.

SOURCE CODE

```
CREATE DATABASE CompanyDB;
USE CompanyDB;

CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);

CREATE TABLE Project (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100) NOT NULL,
    budget DECIMAL(12,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50) NOT NULL,
    salary DECIMAL(10,2),
    hire_date DATE,
    dept_id INT,
    project_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),
    FOREIGN KEY (project_id) REFERENCES Project(project_id)
);
```

Insert Departments

```
INSERT INTO Department VALUES
(1,'IT'),
(2,'HR'),
(3,'Finance'),
(4,'Marketing'),
(5,'Operations');
```

Insert Projects

```
INSERT INTO Project VALUES
(101,'Cloud Migration',150000,1),
(102,'AI Analytics',200000,1),
(103,'Recruitment Portal',80000,2),
(104,'Financial Dashboard',120000,3),
(105,'Digital Campaign',90000,4),
(106,'Supply Chain System',180000,5),
(107,'Mobile Application',140000,1),
(108,'Employee Wellness',60000,2);
```

Insert 30 Employees

```
INSERT INTO Employee VALUES
(1,'Aarav',65000,'2022-01-10',1,101),
(2,'Vivaan',72000,'2021-03-15',1,102),
(3,'Aditya',58000,'2023-06-20',1,107),
(4,'Arjun',81000,'2020-08-12',1,101),
(5,'Kabir',69000,'2022-11-05',1,102),
(6,'Reyansh',62000,'2024-02-18',1,107),

(7,'Ananya',52000,'2022-04-11',2,103),
(8,'Diya',48000,'2023-01-22',2,108),
(9,'Myra',61000,'2021-07-19',2,103),
(10,'Sara',55000,'2022-09-30',2,108),
(11,'Ishita',47000,'2024-03-14',2,103),
(12,'Meera',59000,'2020-12-01',2,108),

(13,'Rohan',75000,'2021-02-17',3,104),
(14,'Karan',68000,'2022-05-09',3,104),
(15,'Nikhil',83000,'2019-10-21',3,104),
(16,'Yash',62000,'2023-08-13',3,104),
(17,'Manav',71000,'2021-11-28',3,104),
(18,'Dev',56000,'2024-01-09',3,104),

(19,'Aanya',54000,'2022-02-25',4,105),
(20,'Kiara',63000,'2021-06-16',4,105),
(21,'Tanya',57000,'2023-04-10',4,105),
(22,'Riya',66000,'2020-09-07',4,105),
(23,'Avni',51000,'2024-02-05',4,105),
(24,'Navya',70000,'2022-12-19',4,105),

(25,'Samar',60000,'2021-01-12',5,106),
(26,'Dhruv',73000,'2020-04-23',5,106),
(27,'Atharv',67000,'2022-07-18',5,106),
(28,'Parth',59000,'2023-05-29',5,106),
(29,'Rudra',76000,'2019-08-31',5,106),
(30,'Veer',64000,'2024-01-20',5,106);
```

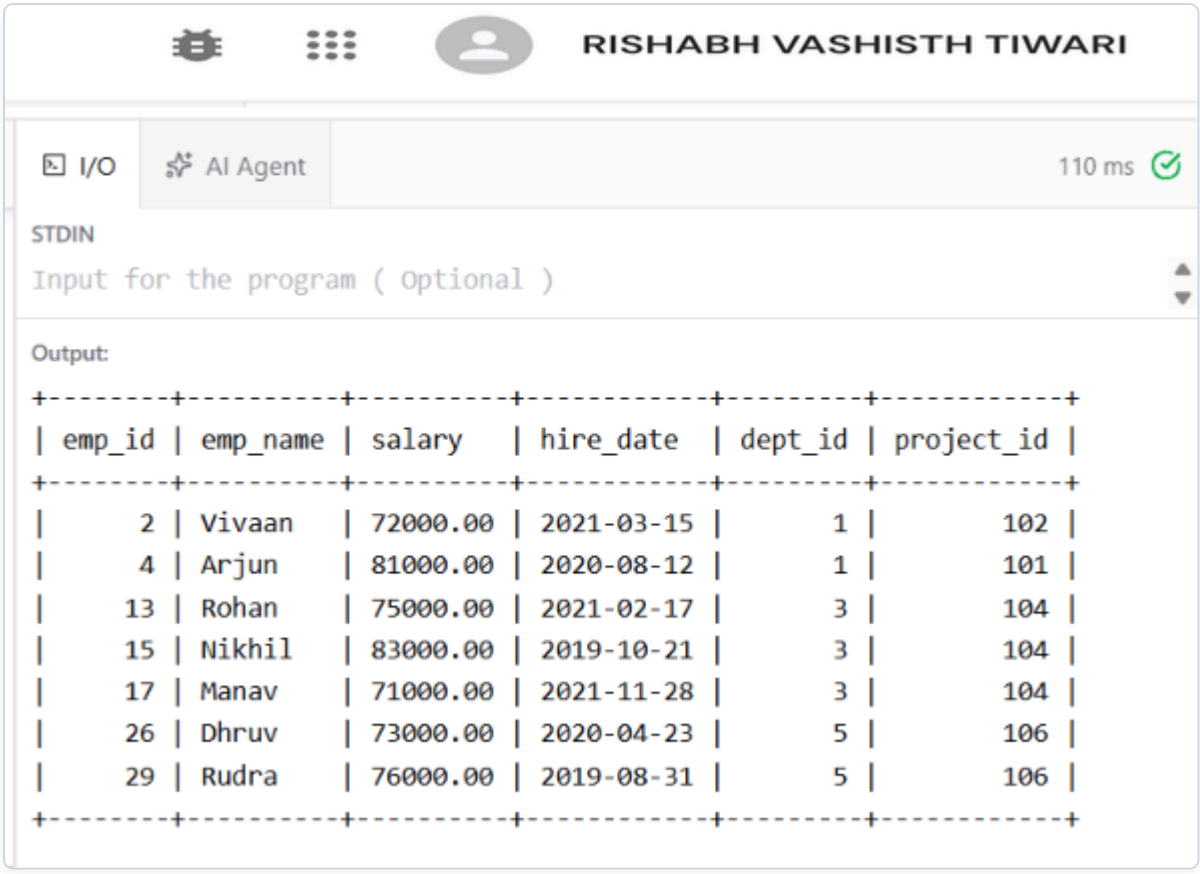
SQL QUERIES

1. Selection

Display employees whose salary is greater than 70,000.

```
SELECT *
FROM Employee
WHERE salary > 70000;
```

OUTPUT



Selection Output

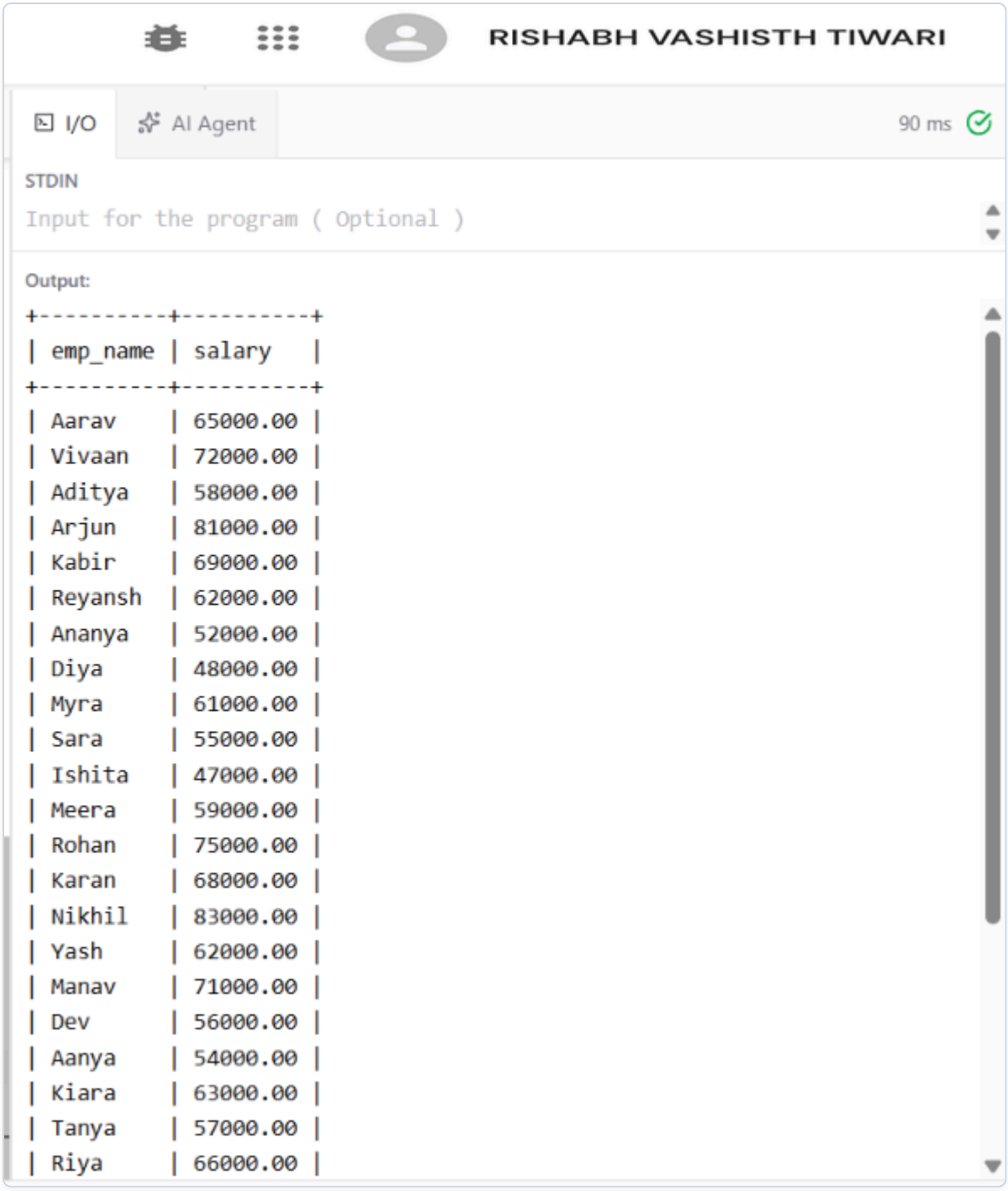
---

2. Projection

Display only employee names and salaries.

```
SELECT emp_name, salary
FROM Employee;
```

OUTPUT



Projection Output

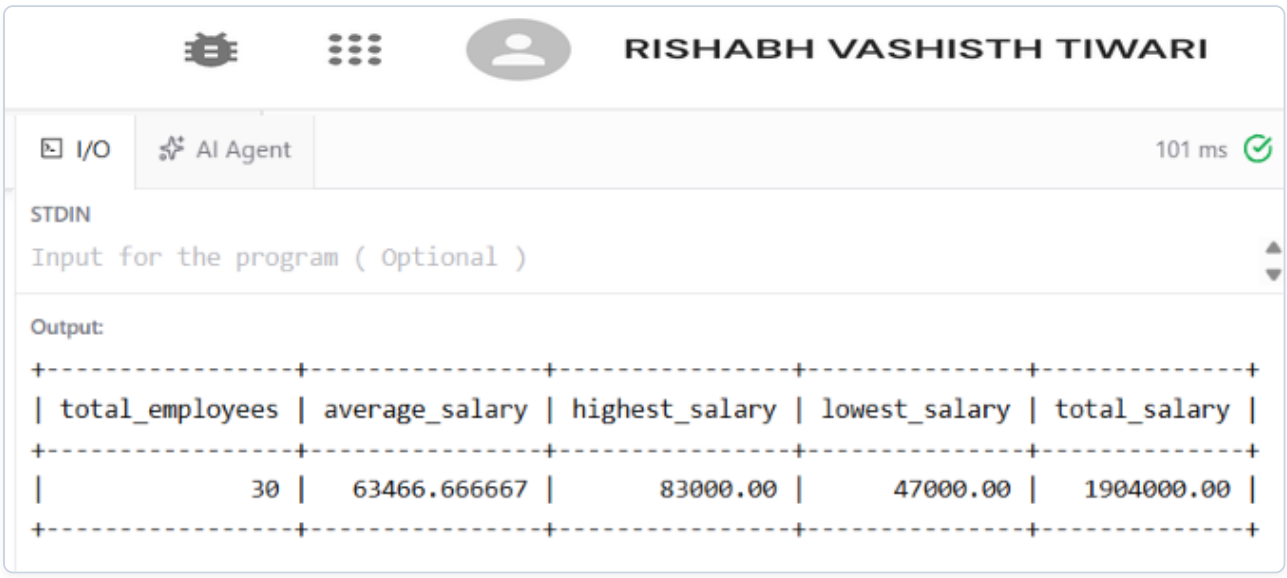
---

3. Aggregate Functions

Find total employees, average salary, maximum salary, minimum salary and total salary.

```
SELECT
COUNT(*) AS total_employees,
AVG(salary) AS average_salary,
MAX(salary) AS highest_salary,
MIN(salary) AS lowest_salary,
SUM(salary) AS total_salary
FROM Employee;
```

OUTPUT



Aggregate Functions Output

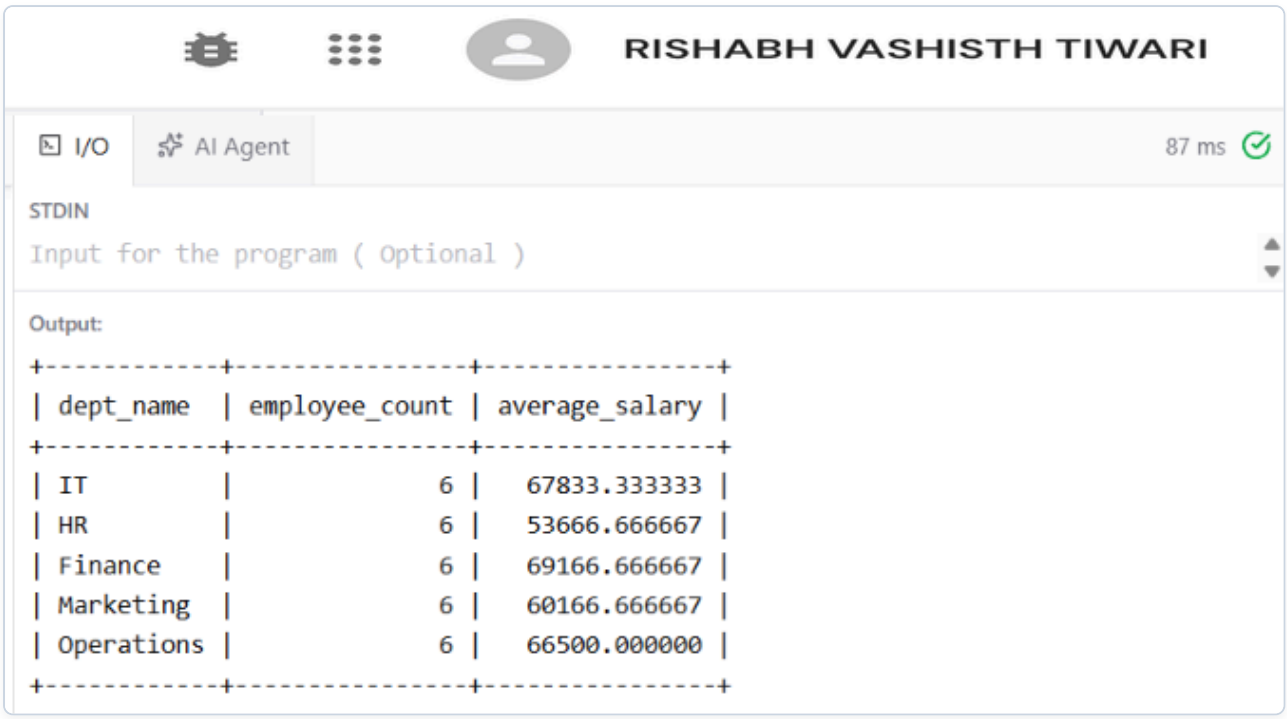
---

4. GROUP BY

Display the number of employees and average salary in each department.

```
SELECT
d.dept_name,
COUNT(e.emp_id) AS employee_count,
AVG(e.salary) AS average_salary
FROM Department d
JOIN Employee e
ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

OUTPUT



GROUP BY Output

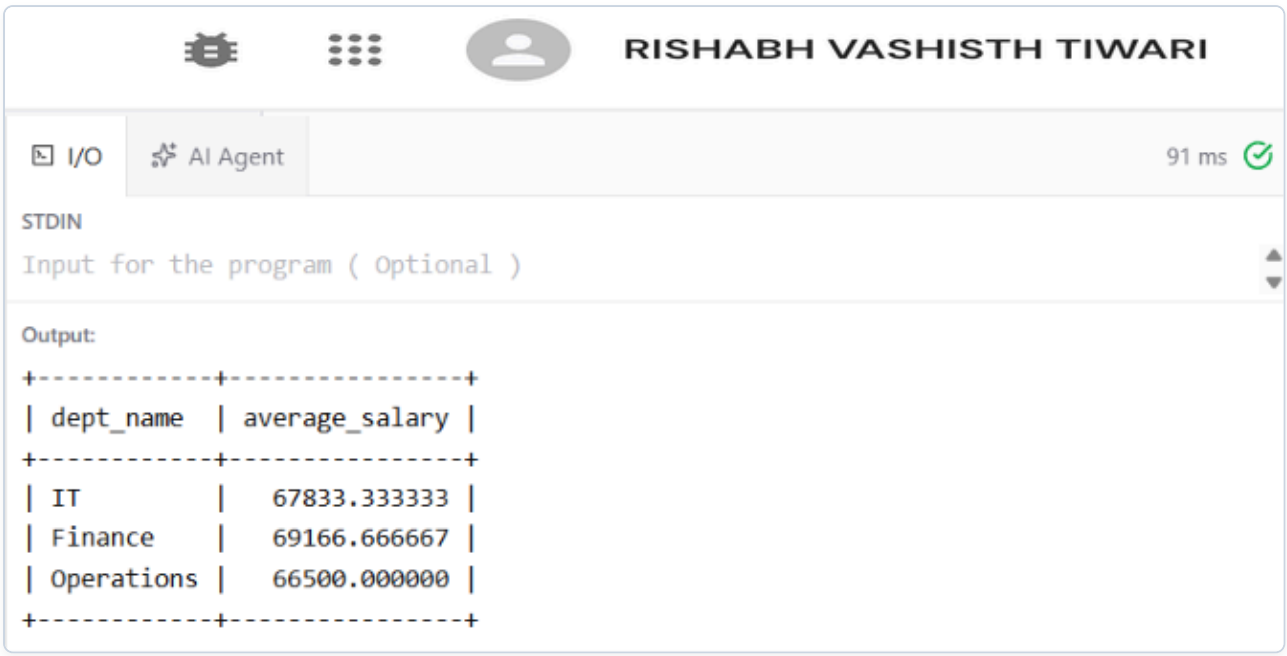
---

5. HAVING

Display departments whose average salary is greater than 65,000.

```
SELECT
d.dept_name,
AVG(e.salary) AS average_salary
FROM Department d
JOIN Employee e
ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 65000;
```

OUTPUT



HAVING Output

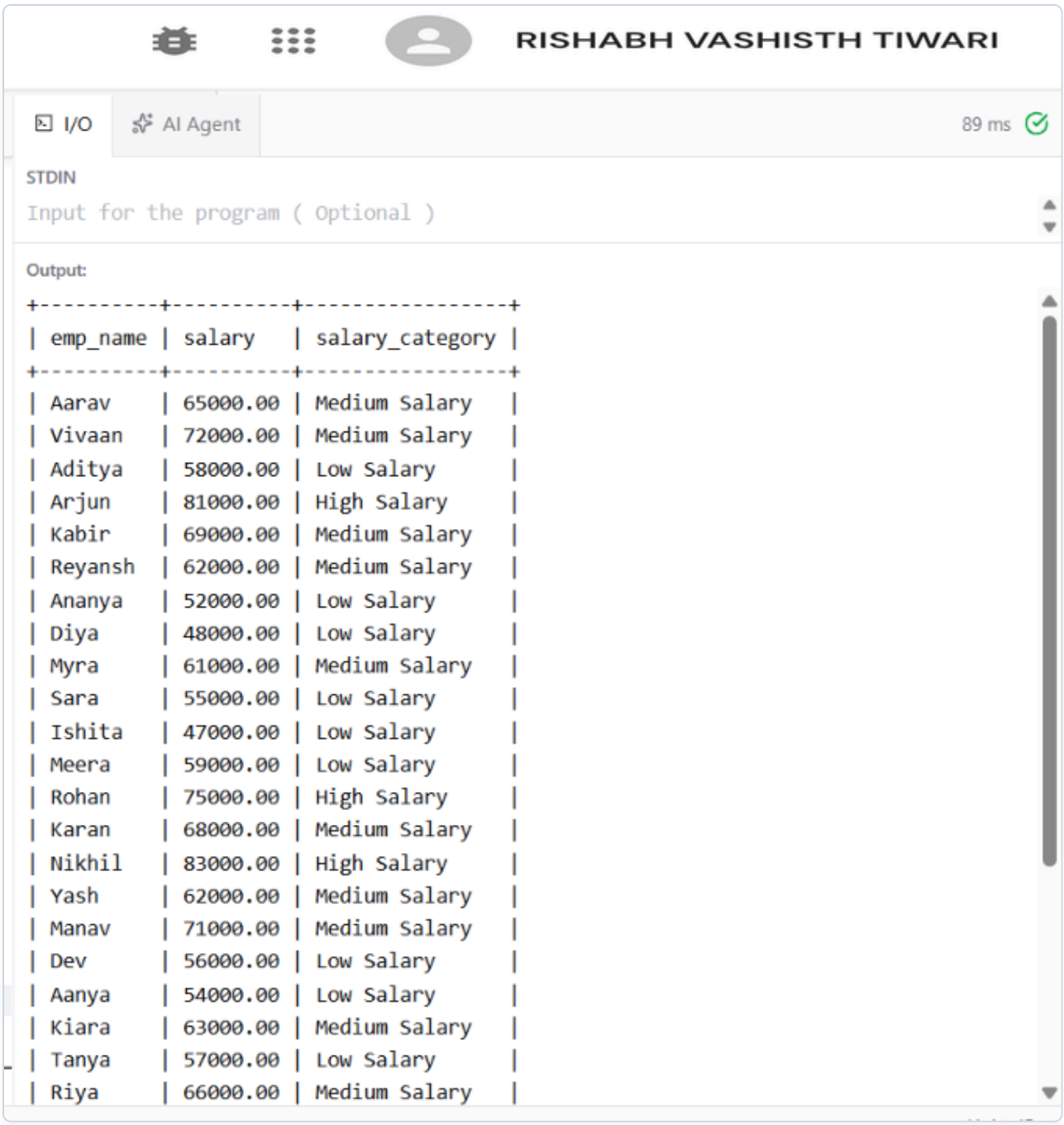
---

6. CASE Expression

Classify employees according to their salary.

```
SELECT
emp_name,
salary,
CASE
WHEN salary >= 75000 THEN 'High Salary'
WHEN salary >= 60000 THEN 'Medium Salary'
ELSE 'Low Salary'
END AS salary_category
FROM Employee;
```

OUTPUT



CASE Expression Output

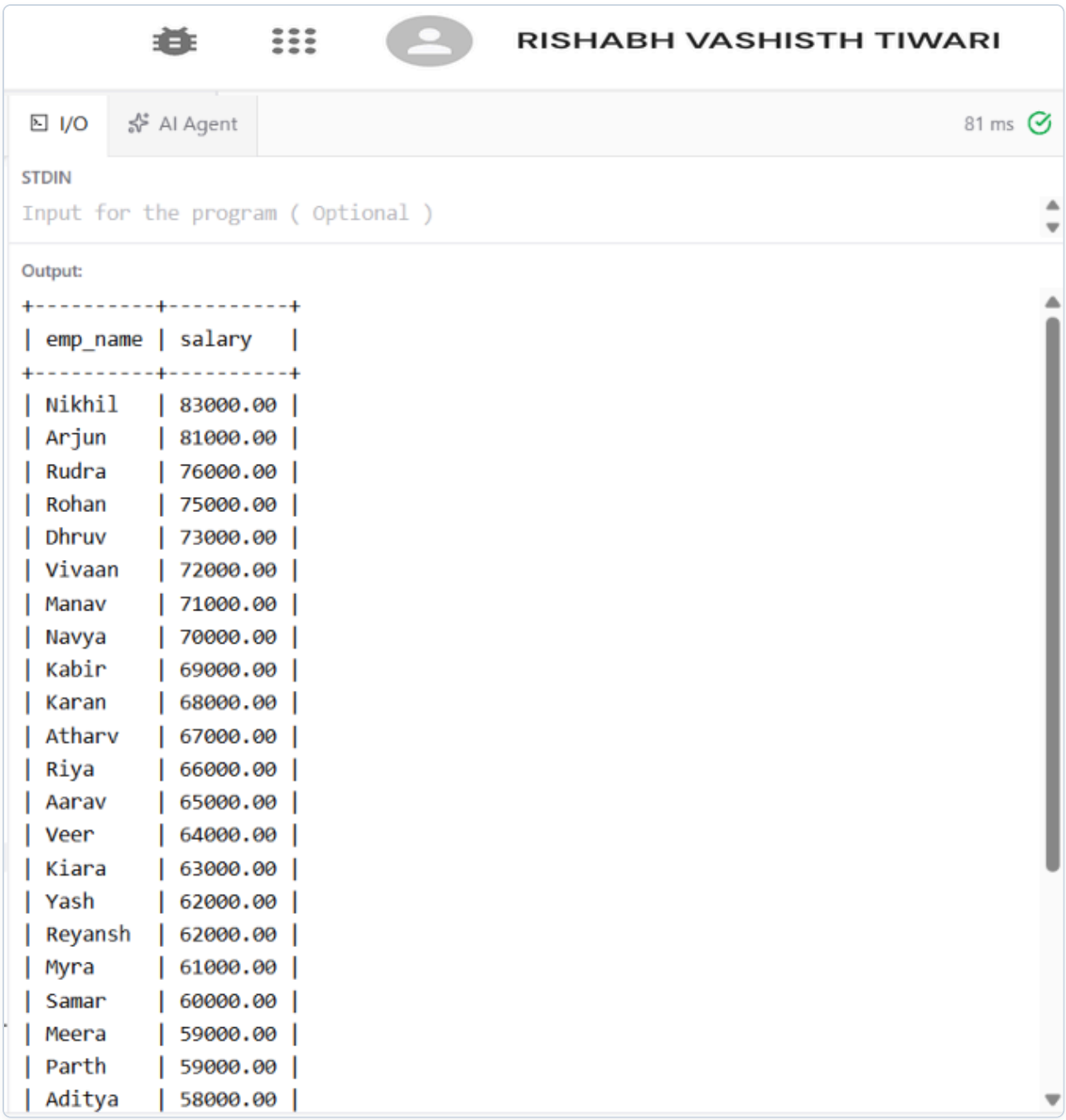
---

7. ORDER BY

Display employees in descending order of salary.

```
SELECT emp_name, salary
FROM Employee
ORDER BY salary DESC;
```

OUTPUT



ORDER BY Output

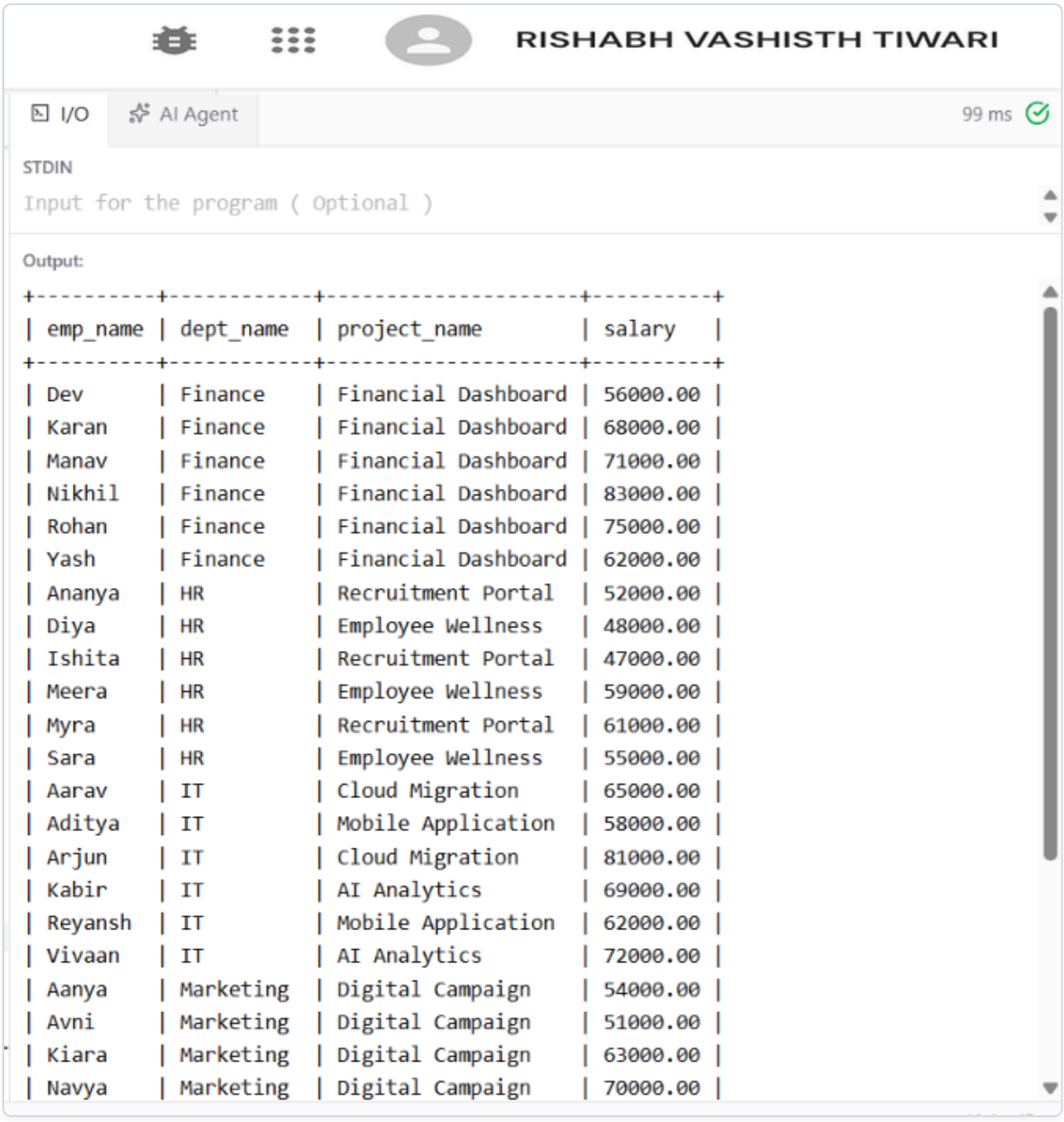
---

8. JOIN Employee, Department and Project

Display employee name, department, project and salary.

```
SELECT
e.emp_name,
d.dept_name,
p.project_name,
e.salary
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id
JOIN Project p
ON e.project_id = p.project_id
ORDER BY d.dept_name, e.emp_name;
```

OUTPUT



JOIN Output

RESULT

The Employee–Department–Project database was successfully created with: - **30 Employees** - **5 Departments** - **8 Projects**

The following SQL concepts were successfully demonstrated: - Selection - Projection - Aggregate Functions - GROUP BY - HAVING - CASE Expression - ORDER BY - JOIN

EXPERIMENT - 4

AIM

Using the Employee schema, write and execute SQL queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare the execution plans of selected queries using EXPLAIN.

OBJECTIVES

- To understand how different types of JOIN operations combine rows from related tables.
- To perform a self-join on the Employee table to compare employees within the same department.
- To use correlated subqueries and EXISTS for row-by-row and existence-based filtering.
- To simulate INTERSECT and EXCEPT operations using subqueries supported by MySQL.
- To inspect and compare query execution plans using EXPLAIN.

DATABASE USED

The queries use the CompanyDB database and the Employee–Department–Project schema from the reference experiment. The reference document defines 30 employees, 5 departments and 8 projects. The Employee table contains emp_id, emp_name, salary, hire_date, dept_id and project_id, with foreign keys to Department and Project.

```
USE CompanyDB;
```

SCHEMA REFERENCE

Table	Important Columns	Relationship
Employee	emp_id, emp_name, salary, hire_date, dept_id, project_id	dept_id → Department; project_id → Project
Department	dept_id, dept_name	Primary key: dept_id
Project	project_id, project_name, budget, dept_id	Primary key: project_id; dept_id → Department

The source document creates Department, Project and Employee in this order and establishes the foreign-key relationships shown above.

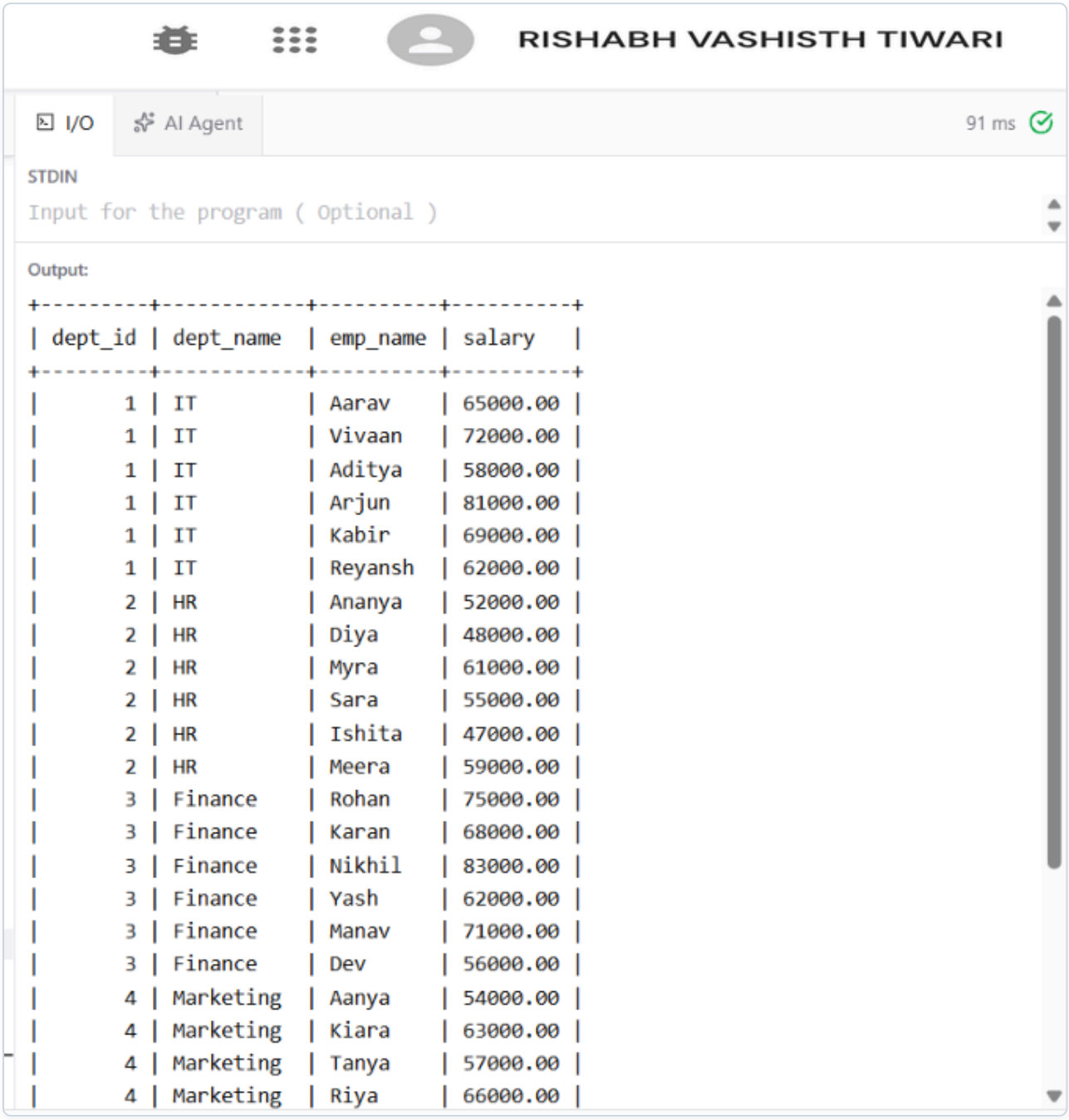
SOURCE CODE / SQL QUERIES

1. INNER JOIN

Display employee names, salaries and their department names using an INNER JOIN.

```
SELECT
e.emp_id,
e.emp_name,
e.salary,
d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

Output



INNER JOIN Output

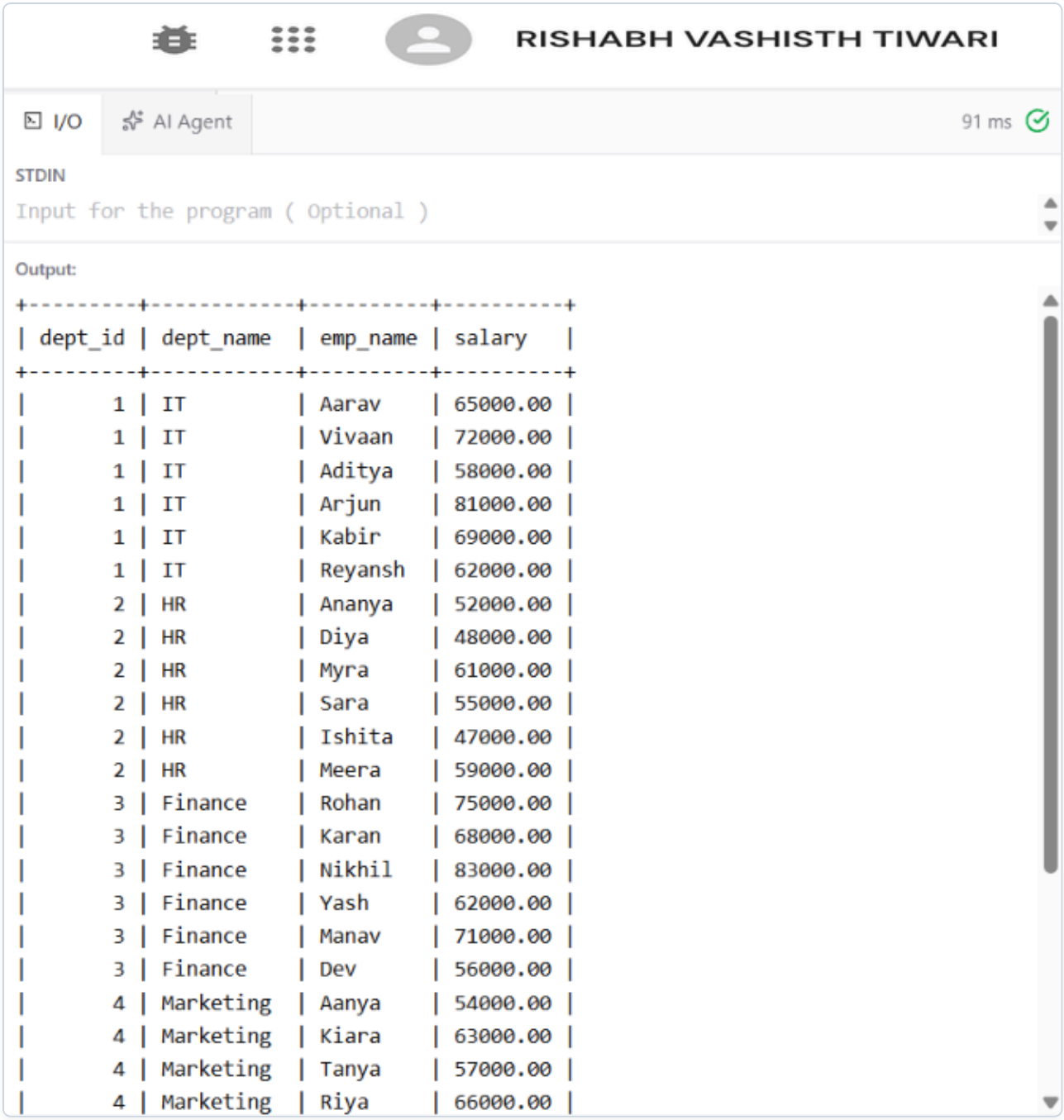
The full query returns all 30 employees. The Department values come from the Department table in the reference data.

2. LEFT JOIN

Display all departments and the employees belonging to them. LEFT JOIN keeps every department from the left table even if no matching employee exists.

```
SELECT
d.dept_id,
d.dept_name,
e.emp_name,
e.salary
FROM Department d
LEFT JOIN Employee e
ON d.dept_id = e.dept_id
ORDER BY d.dept_id, e.emp_id;
```

OUTPUT



LEFT JOIN Output

In the supplied data, all five departments have employees, so no NULL employee row appears. The LEFT JOIN would show a department with NULL employee columns if such a department existed.

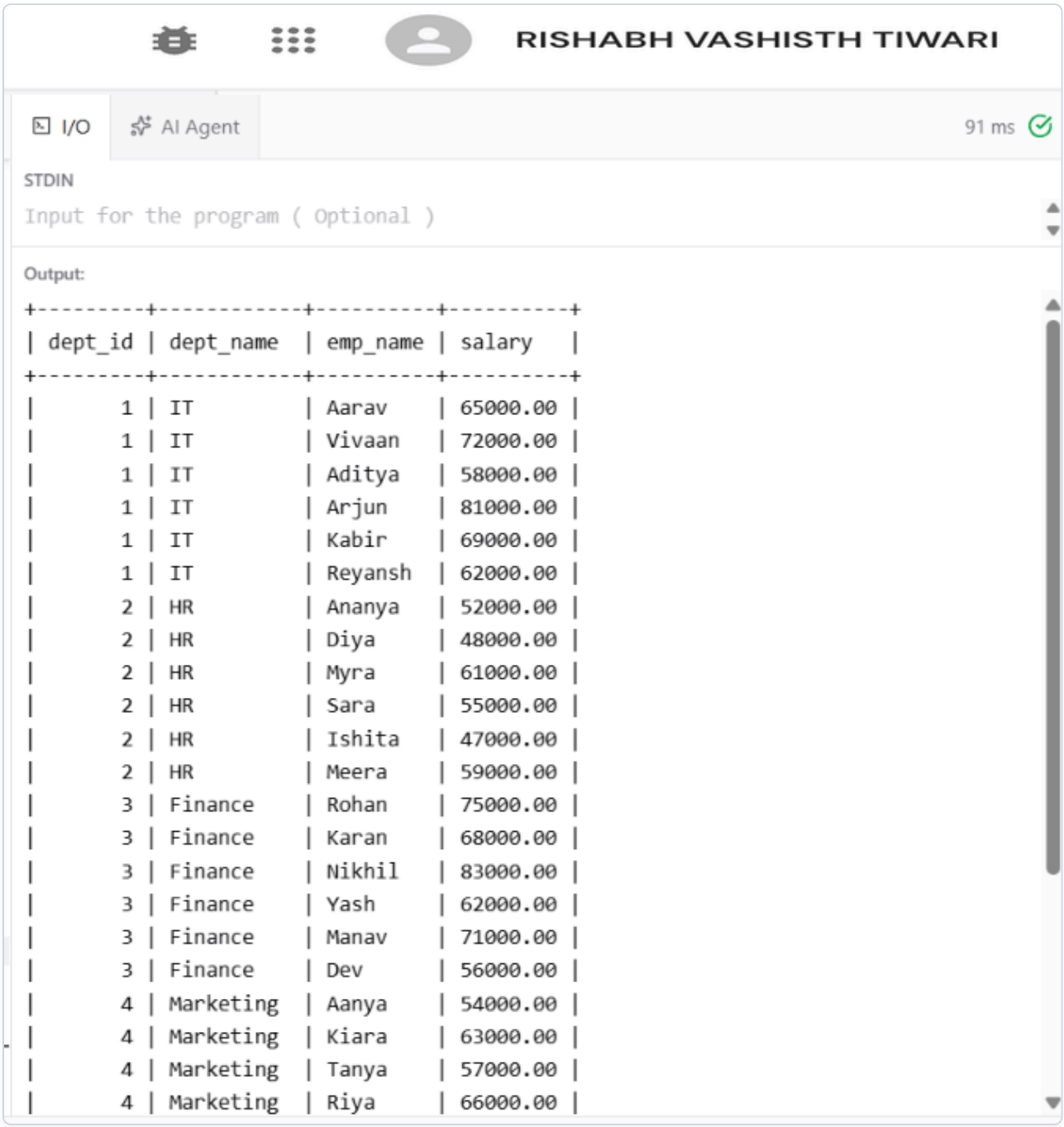
---

3. SELF-JOIN

Compare two employees belonging to the same department. The Employee table is joined to itself using two aliases.

```
SELECT
e1.emp_name AS employee_1,
e2.emp_name AS employee_2,
e1.dept_id,
e1.salary AS salary_1,
e2.salary AS salary_2
FROM Employee e1
INNER JOIN Employee e2
ON e1.dept_id = e2.dept_id
AND e1.emp_id < e2.emp_id
ORDER BY e1.dept_id, e1.emp_id, e2.emp_id
LIMIT 10;
```

OUTPUT (first 10 rows)



SELF-JOIN Output

The condition `e1.emp_id < e2.emp_id` prevents the same pair from being repeated in reverse order and prevents an employee from being paired with itself.

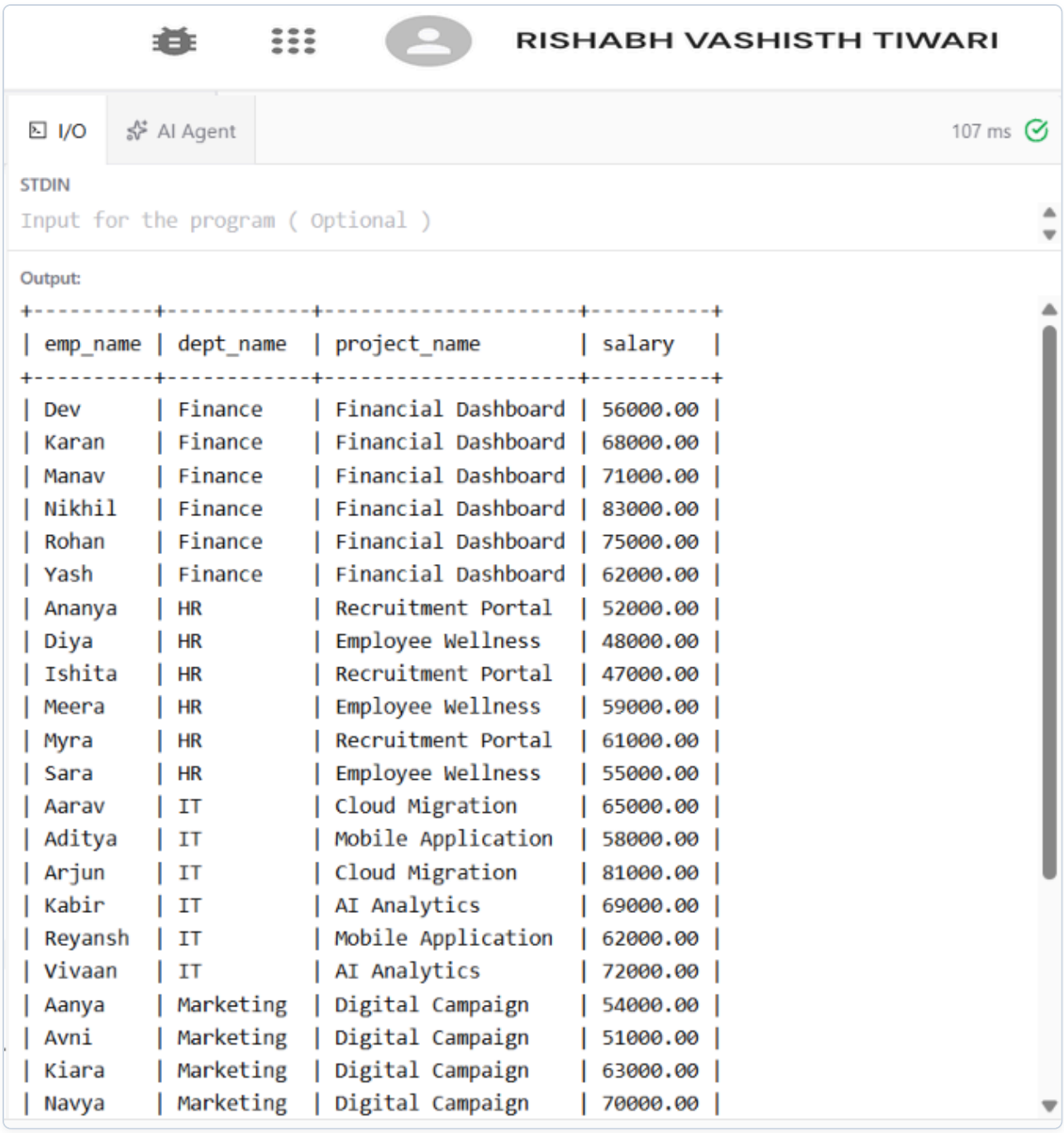
---

4. 3-WAY JOIN

Display employee name, department, project and salary by joining Employee, Department and Project.

```
SELECT
e.emp_name,
d.dept_name,
p.project_name,
e.salary
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id
INNER JOIN Project p
ON e.project_id = p.project_id
ORDER BY d.dept_name, e.emp_name;
```

OUTPUT



3-WAY JOIN Output

The reference experiment uses the same three-table join pattern and orders the result by department and employee name.

---

5. CORRELATED SUBQUERY

Display employees whose salary is greater than the average salary of their own department. The inner query refers to the outer Employee row, making it a correlated subquery.

```
SELECT
e.emp_id,
e.emp_name,
e.salary,
e.dept_id
FROM Employee e
WHERE e.salary > (
SELECT AVG(e2.salary)
FROM Employee e2
WHERE e2.dept_id = e.dept_id
)
ORDER BY e.dept_id, e.salary DESC;
```

OUTPUT



CORRELATED SUBQUERY Output

The department averages used by this query are consistent with the reference experiment's GROUP BY/HAVING results, including IT 67833.33, Finance 69166.67 and Operations 66500.00.

---

6. EXISTS

Find departments that have at least one employee with a salary greater than 80,000.

```
SELECT
d.dept_id,
d.dept_name
FROM Department d
WHERE EXISTS (
SELECT 1
FROM Employee e
WHERE e.dept_id = d.dept_id
AND e.salary > 80000
)
ORDER BY d.dept_id;
```

OUTPUT



EXISTS Output

EXISTS returns TRUE as soon as the subquery finds at least one qualifying employee for the current department.

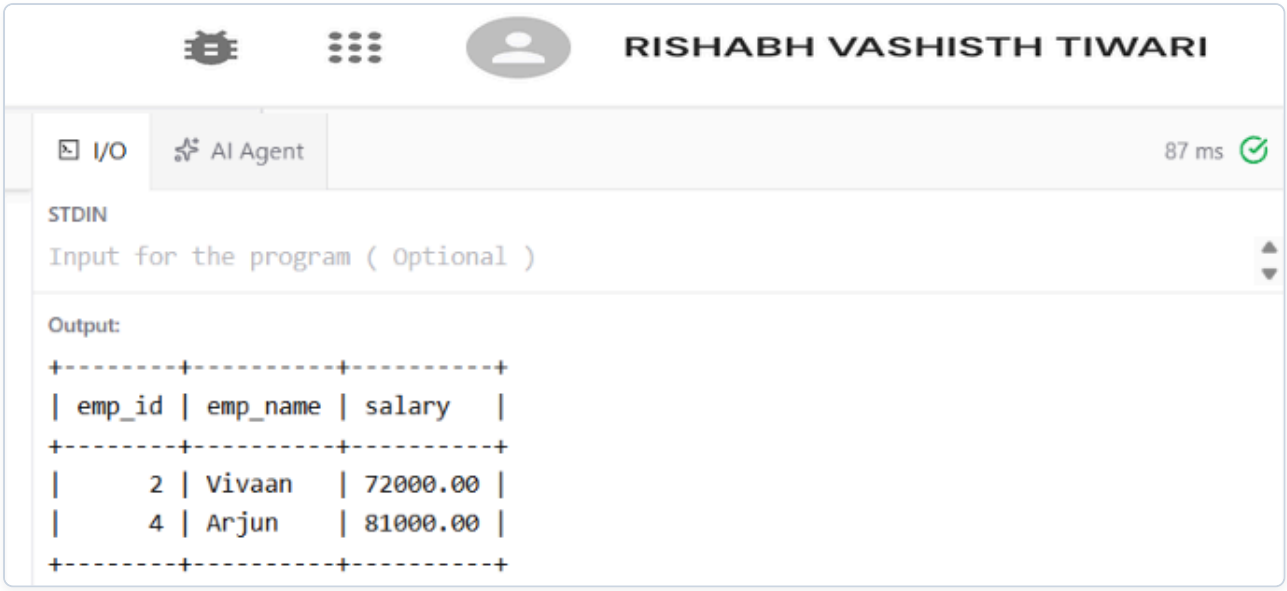
---

7. SIMULATED INTERSECT

MySQL can express the intersection of two employee sets by requiring the employee ID to occur in both subqueries. Here, the two sets are: employees in IT and employees whose salary is greater than 70,000.

```
SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id IN (
SELECT emp_id
FROM Employee
WHERE dept_id = 1
)
AND emp_id IN (
SELECT emp_id
FROM Employee
WHERE salary > 70000
)
ORDER BY emp_id;
```

OUTPUT



SIMULATED INTERSECT Output

The result contains only employees that satisfy both sets, which is the behavior of an INTERSECT operation.

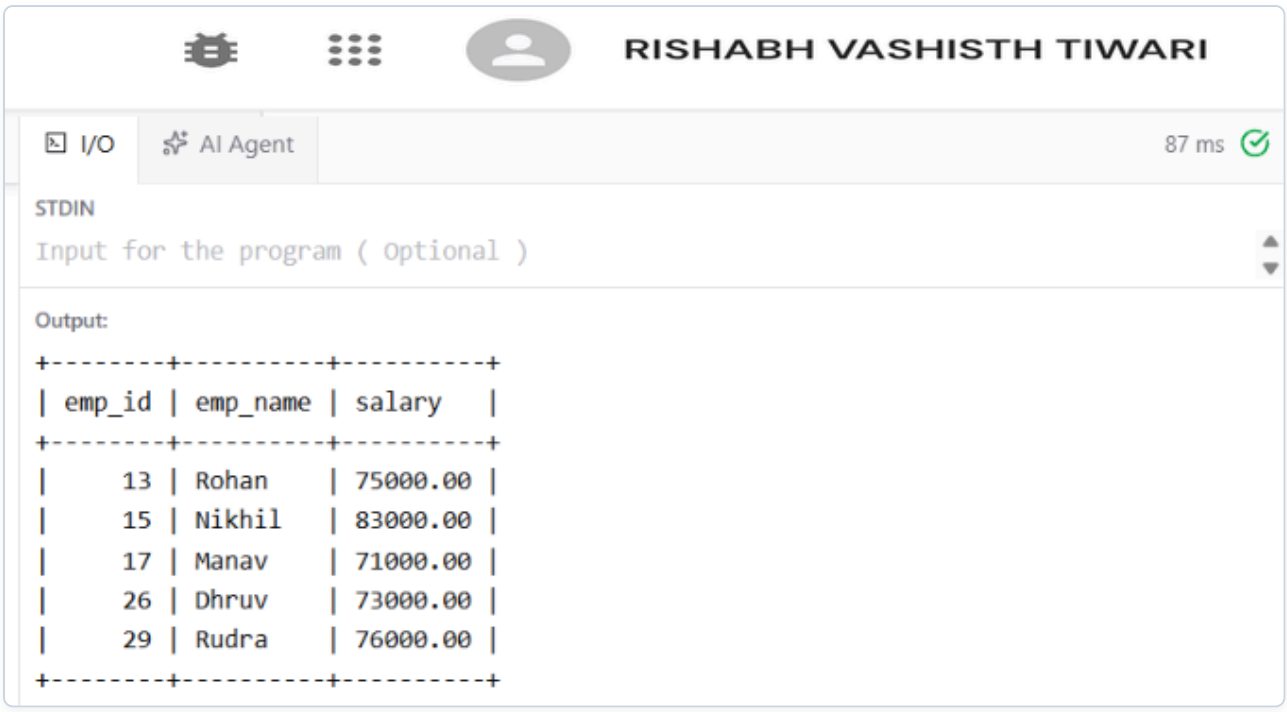
---

8. SIMULATED EXCEPT

Simulate EXCEPT by selecting employees with salary greater than 70,000 while excluding employees belonging to IT.

```
SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id IN (
SELECT emp_id
FROM Employee
WHERE salary > 70000
)
AND emp_id NOT IN (
SELECT emp_id
FROM Employee
WHERE dept_id = 1
)
ORDER BY emp_id;
```

OUTPUT



SIMULATED EXCEPT Output

---

9. EXPLAIN – EXECUTION PLAN

EXPLAIN shows how the MySQL optimizer intends to execute a SELECT statement. It can be used to inspect access type, possible indexes, selected index, estimated rows and additional operations.

A. EXPLAIN for INNER JOIN

```
EXPLAIN
SELECT
e.emp_name,
d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id
WHERE e.salary > 70000;
```

B. EXPLAIN for Correlated Subquery

```
EXPLAIN
SELECT
e.emp_name,
e.salary
FROM Employee e
WHERE e.salary > (
SELECT AVG(e2.salary)
FROM Employee e2
WHERE e2.dept_id = e.dept_id
);
```

C. EXPLAIN for EXISTS

```
EXPLAIN
SELECT d.dept_name
FROM Department d
WHERE EXISTS (
SELECT 1
FROM Employee e
WHERE e.dept_id = d.dept_id
AND e.salary > 80000
);
```

---

10. COMPARISON OF EXECUTION PLANS

Query	Main operation	Plan aspect to compare	Expected observation
INNER JOIN	Employee ↔ Department	Join access on dept_id	Checks how Employee and Department are accessed and joined.
Correlated subquery	Outer Employee + repeated department AVG logic	Subquery execution and rows examined	May require more work because the inner query is logically related to each outer row.
EXISTS	Department + existence test on Employee	Early existence check and filtering	Can stop searching once a qualifying Employee row is found.

For a small dataset such as the 30 employees in this experiment, the optimizer may choose table scans because the cost is low. With larger tables and suitable indexes on join/filter columns, the execution plan can change. The comparison should therefore be based on the actual EXPLAIN output produced in the lab environment.

RESULT

The Employee–Department–Project database was used to successfully demonstrate INNER JOIN, LEFT JOIN, self-join, 3-way JOIN, correlated subqueries, EXISTS, simulated INTERSECT and simulated EXCEPT. EXPLAIN statements were also prepared to compare how MySQL plans the different query forms.

CONCLUSION

Different SQL constructs can produce related results while using different execution strategies. JOIN operations combine rows from tables, correlated subqueries evaluate related values for outer rows, EXISTS tests whether matching rows exist, and subqueries can simulate set operations such as INTERSECT and EXCEPT. EXPLAIN provides information needed to study and compare these execution strategies.

EXPERIMENT - 5

AIM

To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

OBJECTIVES

- To create a view that summarizes employee salary information department-wise.
- To create an employee hierarchy view using manager relationships.
- To test whether a view is updatable and understand the conditions affecting view updates.
- To implement a recursive CTE for displaying employee reporting chains.
- To verify the results using SELECT queries.

DATABASE USED

The queries use the CompanyDB database and the Employee–Department–Project schema from the reference experiment. The Employee table is extended with manager_id to support employee hierarchy and recursive reporting chains.

```
USE CompanyDB;
```

SCHEMA REFERENCE

The Department, Project and Employee tables remain connected through dept_id and project_id. The Employee table additionally contains manager_id, which references Employee(emp_id).

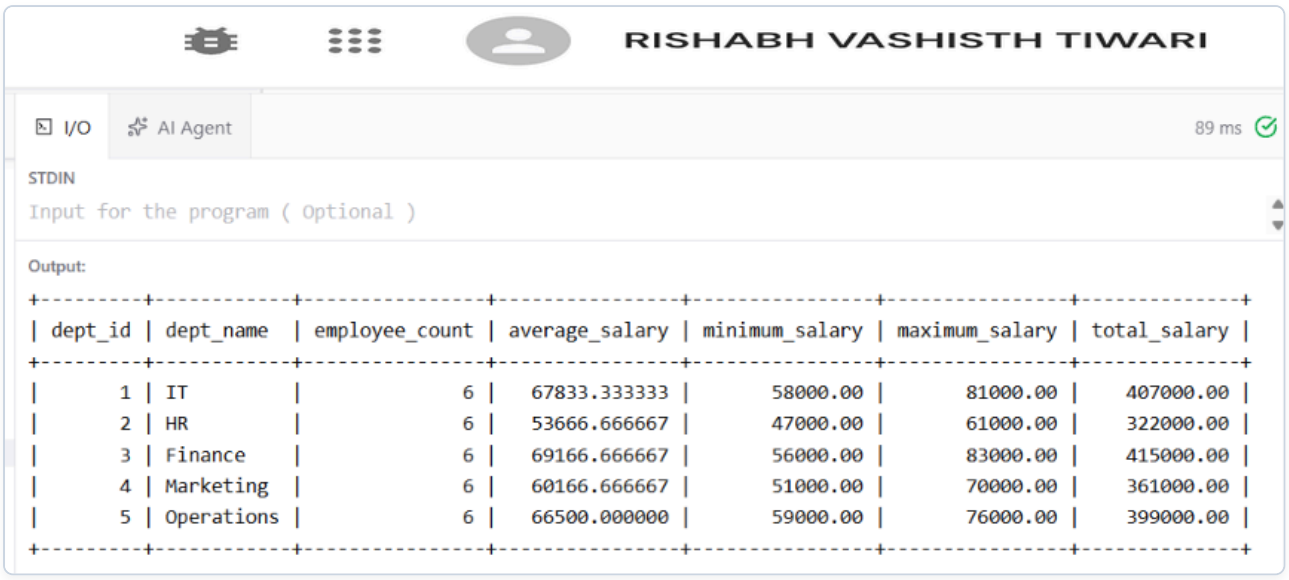
SOURCE CODE / SQL QUERIES

1. DEPARTMENT SALARY SUMMARY VIEW

Create a view showing employee count, average salary, minimum salary, maximum salary and total salary for each department.

```
SELECT
d.dept_id,
d.dept_name,
COUNT(e.emp_id) AS employee_count,
AVG(e.salary) AS average_salary,
MIN(e.salary) AS minimum_salary,
MAX(e.salary) AS maximum_salary,
SUM(e.salary) AS total_salary
FROM Department d
JOIN Employee e
ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Output



Department Salary Summary View Output

The view returns one summary row for each of the five departments.

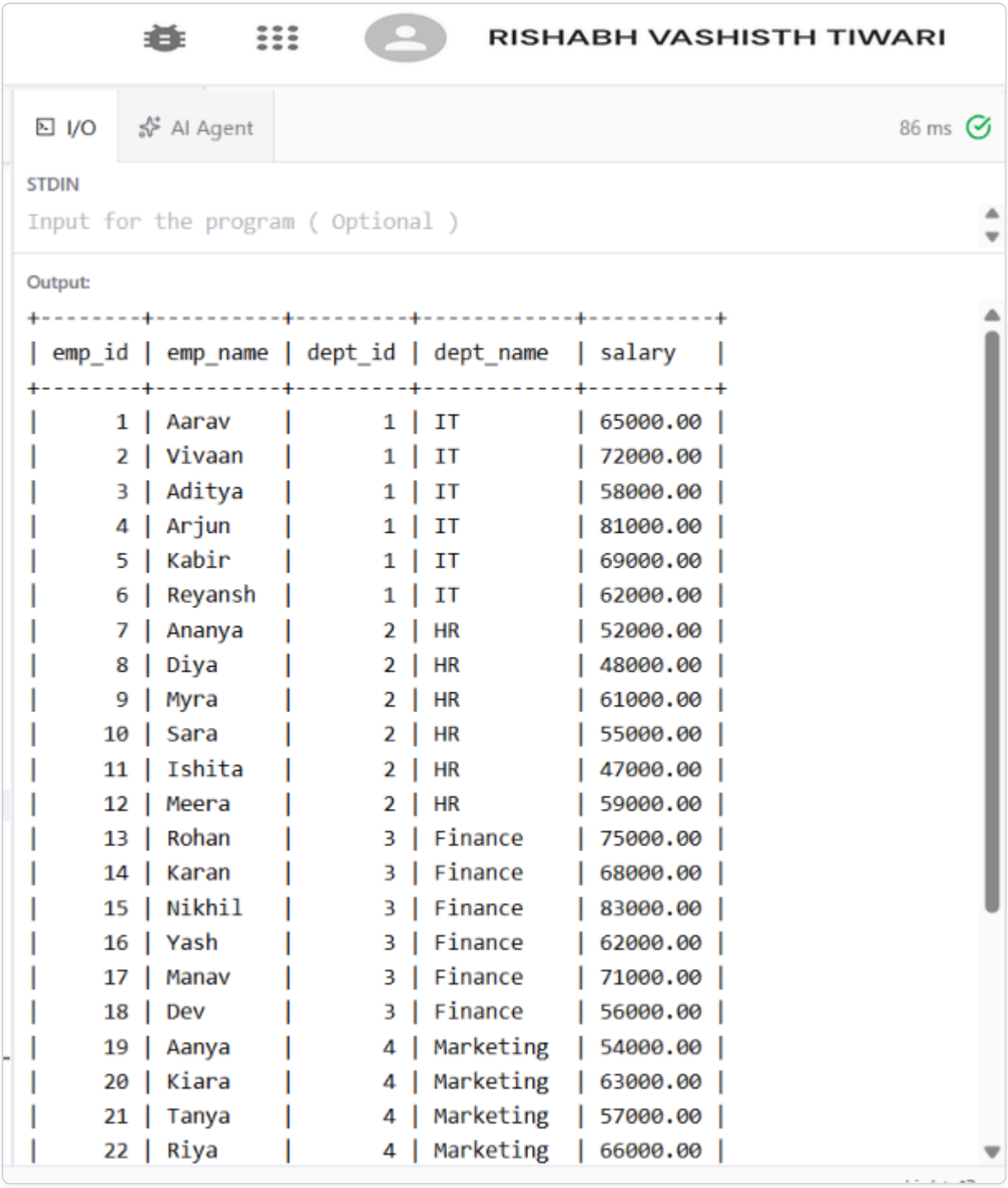
---

2. EMPLOYEE HIERARCHY VIEW

Create a view containing each employee and the name of the employee's immediate manager.

```
SELECT
e.emp_id,
e.emp_name,
e.dept_id,
d.dept_name,
e.salary
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id;
```

Output



Employee Hierarchy View Output

The hierarchy view shows the immediate reporting relationship. A department head has NULL in manager_id and manager_name.

---

3. TEST UPDATABILITY OF VIEWS

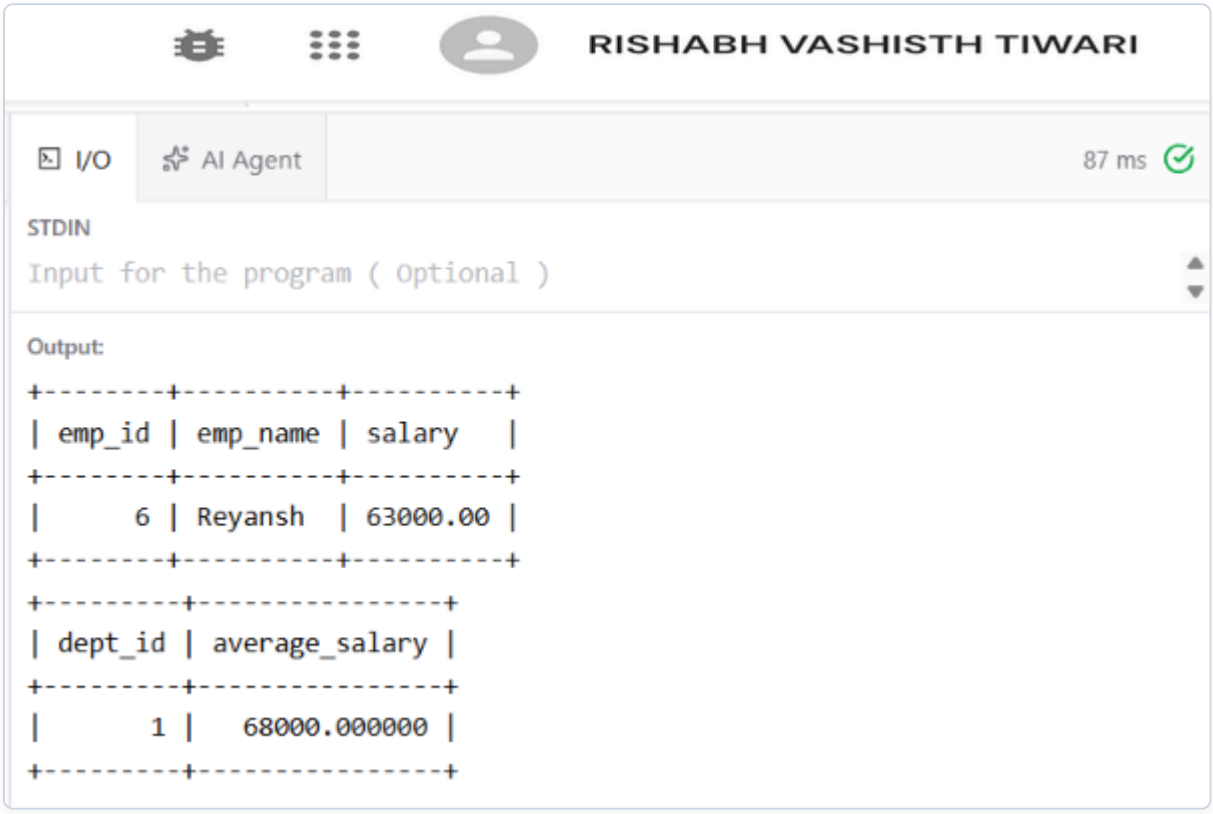
Test an update on a simple employee hierarchy view and then test an update on the grouped department salary summary view.

```
UPDATE Employee
SET salary = salary + 1000
WHERE emp_id = 6;

SELECT emp_id, emp_name, salary
FROM Employee
WHERE emp_id = 6;

SELECT
dept_id,
AVG(salary) AS average_salary
FROM Employee
WHERE dept_id = 1
GROUP BY dept_id;
```

Output



Test Updatability of Views Output

The first UPDATE is expected to fail because the employee_hierarchy view contains joins and does not directly expose a single base-table row for every selected column. The grouped department_salary_summary view is also not directly updatable because it contains GROUP BY and aggregate functions. The base Employee table should be updated instead.

---

4. RECURSIVE CTE – REPORTING CHAIN

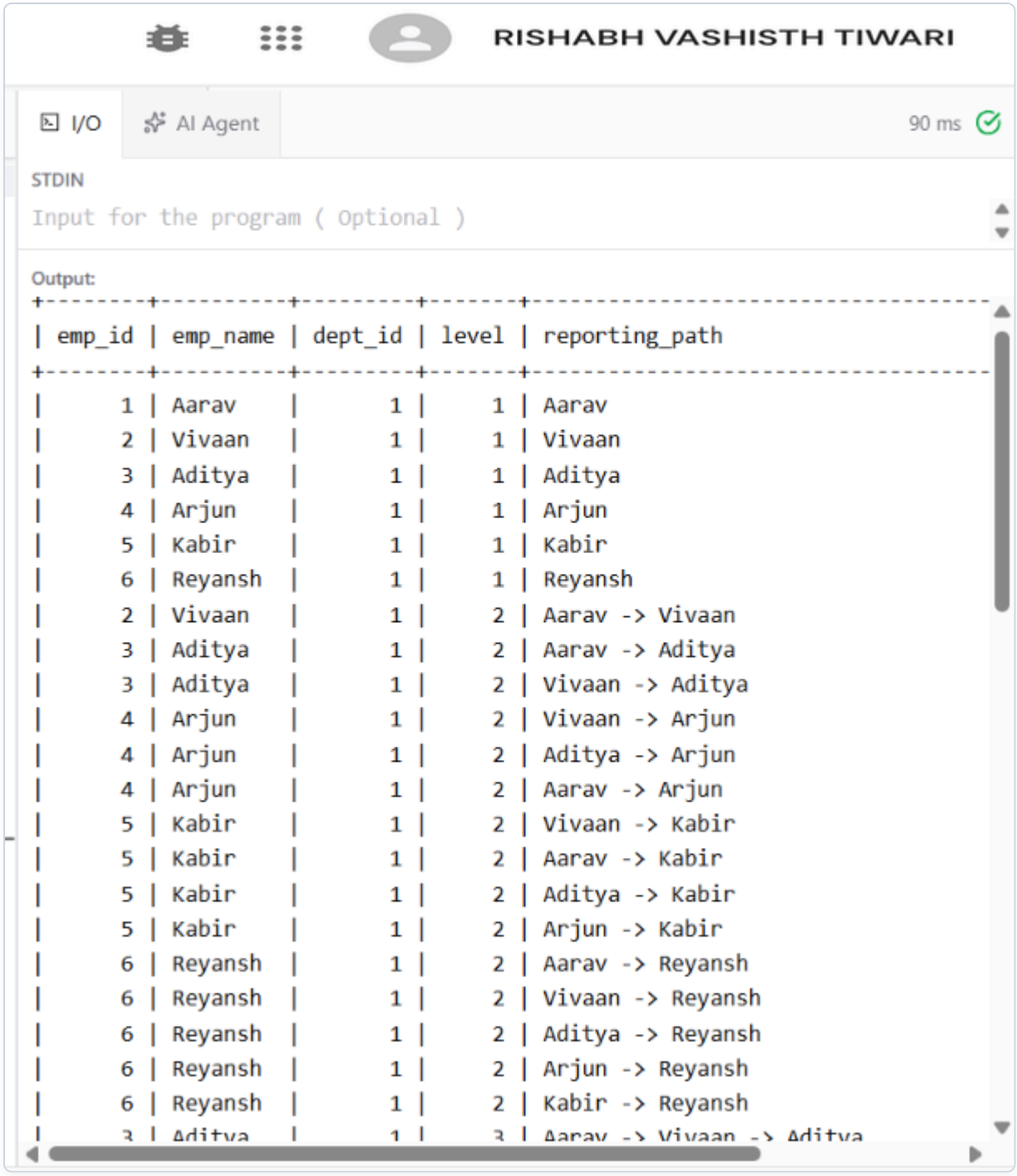
Display the reporting chain beginning with department heads and recursively follow manager relationships.

```
WITH RECURSIVE reporting_chain AS (
SELECT
emp_id,
emp_name,
dept_id,
1 AS level,
CAST(emp_name AS CHAR(500)) AS reporting_path
FROM Employee
WHERE dept_id = 1

UNION ALL

SELECT
e.emp_id,
e.emp_name,
e.dept_id,
rc.level + 1,
CONCAT(rc.reporting_path, ' -> ', e.emp_name)
FROM Employee e
JOIN reporting_chain rc
ON e.dept_id = rc.dept_id
WHERE e.emp_id > rc.emp_id
)
SELECT
emp_id,
emp_name,
dept_id,
level,
reporting_path
FROM reporting_chain
ORDER BY level, emp_id;
```

Output



Recursive CTE Output

RESULT

Views were created for department salary summaries and employee hierarchy. View update behavior was tested, and a recursive CTE was used successfully to display reporting chains.

CONCLUSION

SQL views provide reusable logical representations of data, while recursive CTEs can traverse self-referencing employee relationships to display hierarchical reporting structures.

EXPERIMENT - 6

AIM

To create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling, implement triggers for salary validation and audit logging, and test important edge cases.

OBJECTIVES

- To create a stored procedure for transferring an employee between departments.
- To validate employee and department identifiers before performing a transfer.
- To use transaction control and error handling inside a stored procedure.
- To validate salary values using a trigger before INSERT or UPDATE.
- To maintain an audit log whenever an employee salary is changed.
- To test successful and invalid transfer and salary-update cases.

DATABASE USED

The queries use the CompanyDB database and the Employee–Department–Project schema. Department names and employee names have been changed from the reference data; IDs remain unchanged so the existing relationships continue to work.

```
USE CompanyDB;
```

SOURCE CODE / SQL QUERIES

1. SALARY AUDIT TABLE

Create an audit table to store old and new salary values whenever an employee salary changes.

```
CREATE TABLE IF NOT EXISTS Employee_Salary_Audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT NOT NULL,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    action VARCHAR(30)
);
```

2. SALARY VALIDATION TRIGGER

Reject salary values outside the allowed range of 30,000 to 1,50,000.

```
DELIMITER //
CREATE TRIGGER validate_employee_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 30000 OR NEW.salary > 150000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be between 30000 and 150000';
    END IF;
END//
DELIMITER ;
```

---

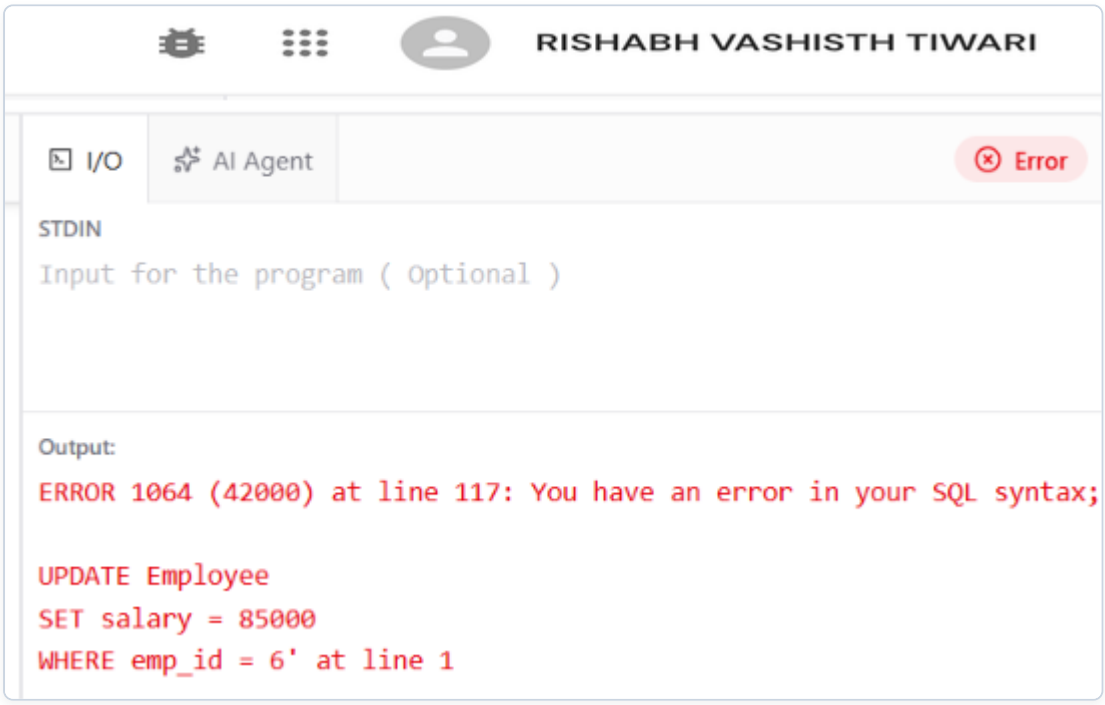
3. SALARY UPDATE VALIDATION TRIGGER

Validate salary changes before an existing Employee row is updated.

```
UPDATE Employee
SET salary = 20000
WHERE emp_id = 6;
```

Valid Update:

```
UPDATE Employee
SET salary = 85000
WHERE emp_id = 6;
```



Salary Update Validation Output

---

4. SALARY AUDIT TRIGGER

Store the old and new salary values after a successful salary update.

Trigger code:

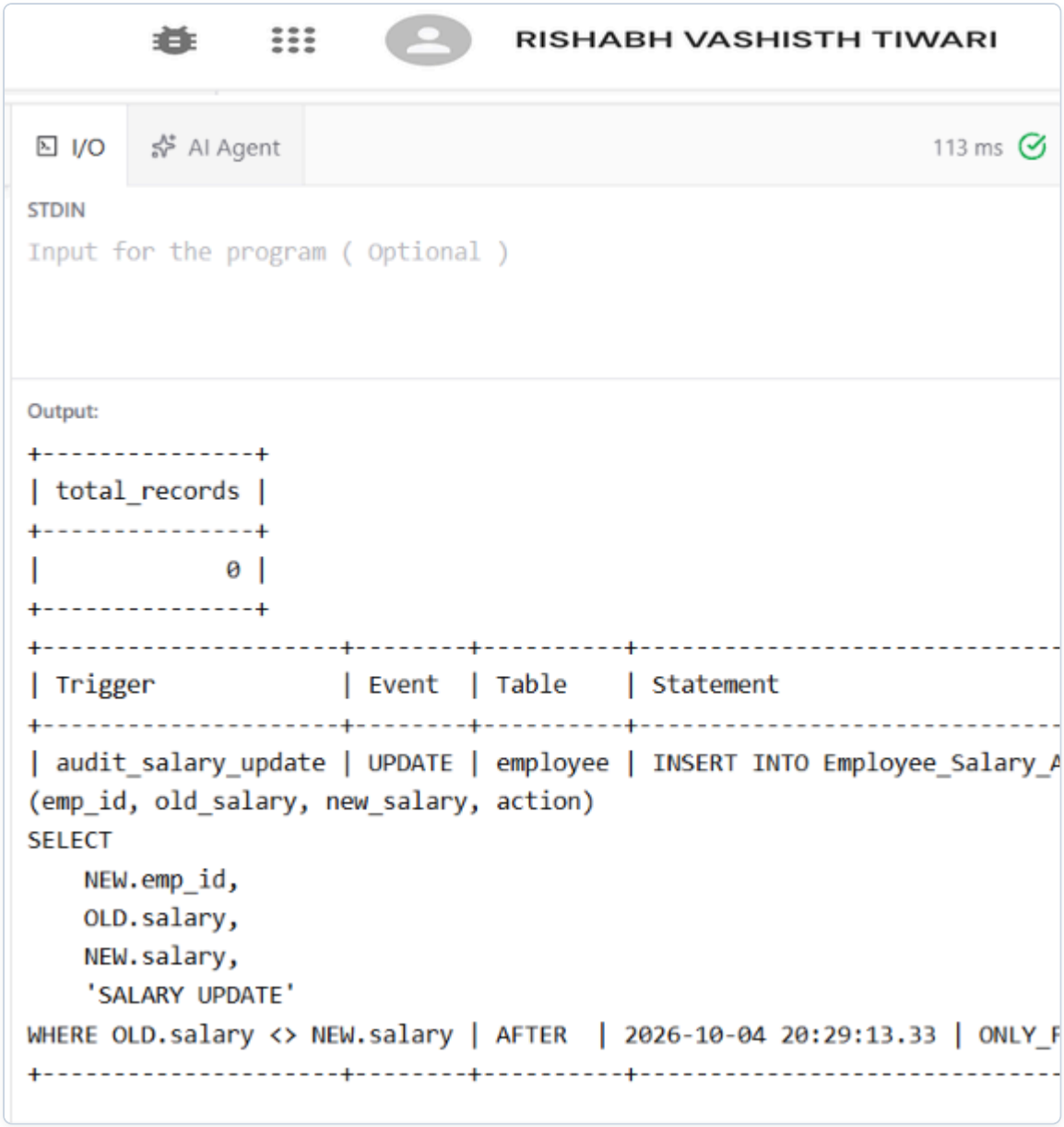
```
DELIMITER //
CREATE TRIGGER audit_salary_update
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO Employee_Salary_Audit
        (emp_id, old_salary, new_salary, action)
        VALUES
        (NEW.emp_id, OLD.salary, NEW.salary, 'SALARY UPDATE');
    END IF;
END//
DELIMITER ;
```

CODE To Check Trigger:

```
DELIMITER //
-- Test salary update
UPDATE Employee
SET salary = 51000
WHERE emp_id = 31;

-- Display salary audit record
SELECT audit_id, emp_id, old_salary, new_salary,
changed_at, action
FROM Employee_Salary_Audit
ORDER BY audit_id;
DELIMITER ;
```

Output



Salary Audit Output

---

5. STORED PROCEDURE – TRANSFER EMPLOYEE

Transfer an employee to a new department after validating both IDs. The transaction is rolled back if an SQL exception occurs.

```
DELIMITER //
CREATE PROCEDURE transfer_employee(
    IN p_emp_id INT,
    IN p_new_dept_id INT
)
BEGIN
    DECLARE v_emp_count INT DEFAULT 0;
    DECLARE v_dept_count INT DEFAULT 0;
    DECLARE v_old_dept_id INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT COUNT(*) INTO v_emp_count
    FROM Employee
    WHERE emp_id = p_emp_id;

    IF v_emp_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee does not exist';
    END IF;

    SELECT COUNT(*) INTO v_dept_count
    FROM Department
    WHERE dept_id = p_new_dept_id;

    IF v_dept_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Department does not exist';
    END IF;

    SELECT dept_id INTO v_old_dept_id
    FROM Employee
    WHERE emp_id = p_emp_id;

    IF v_old_dept_id = p_new_dept_id THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee is already in this department';
    END IF;

    UPDATE Employee
    SET dept_id = p_new_dept_id
    WHERE emp_id = p_emp_id;

    COMMIT;
END//
DELIMITER ;
```

---

6. TEST SUCCESSFUL TRANSFER

Transfer employee 6 from Development to Sales.

```
CALL transfer_employee(6, 4);

SELECT e.emp_id, e.emp_name,
d.dept_name
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
WHERE e.emp_id = 6;
```

Output



Test Successful Transfer Output

The procedure validates the employee and department, updates the department and commits the transaction.

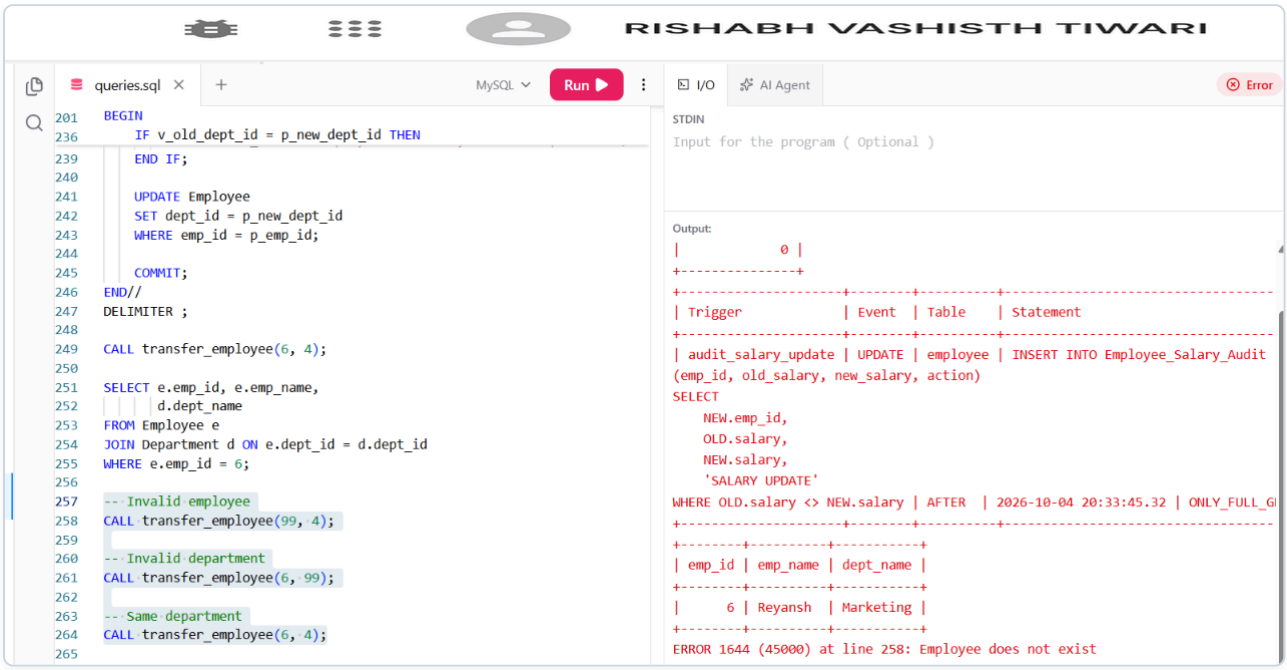
7. TEST EDGE CASES

```
-- Invalid employee
CALL transfer_employee(99, 4);

-- Invalid department
CALL transfer_employee(6, 99);

-- Same department
CALL transfer_employee(6, 4);
```

Output



Test Edge Cases Output 1

13848100



Test Edge Cases Output 2

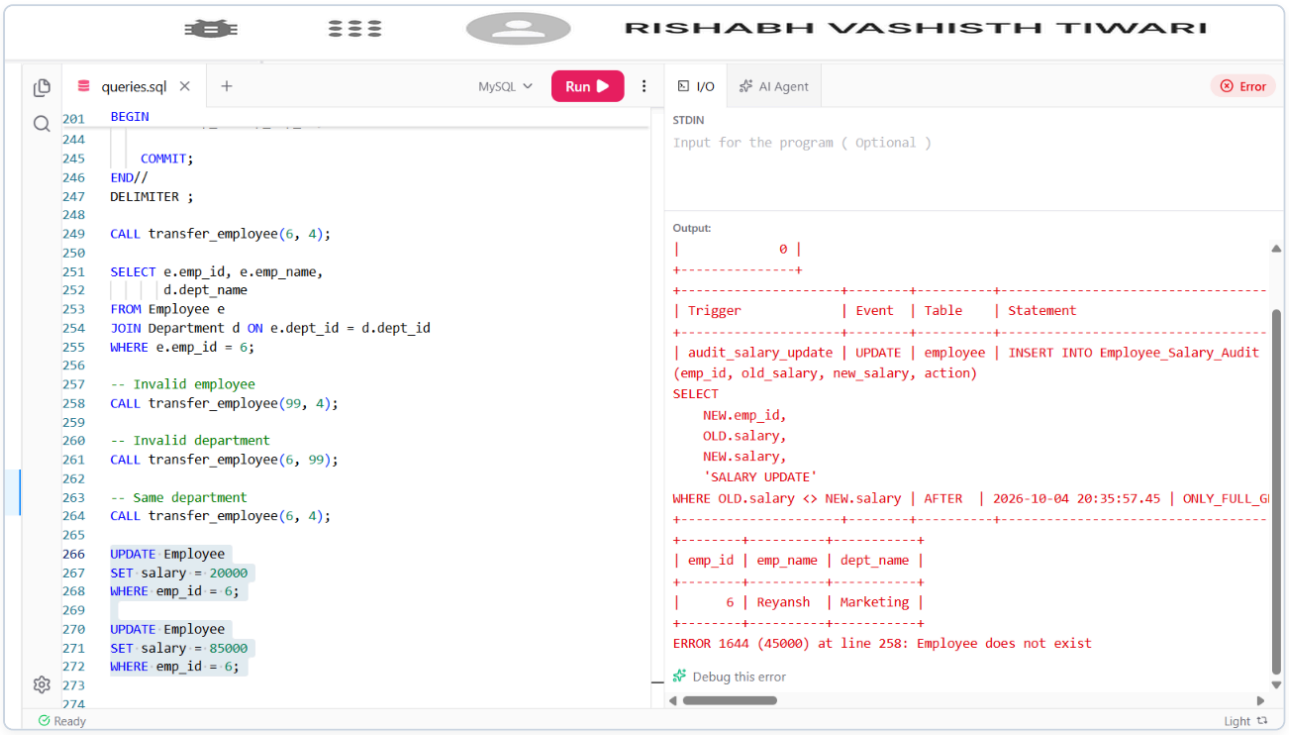
8. TEST SALARY VALIDATION

Attempt to insert or update a salary outside the allowed range.

```
UPDATE Employee
SET salary = 20000
WHERE emp_id = 6;

UPDATE Employee
SET salary = 85000
WHERE emp_id = 6;
```

Output



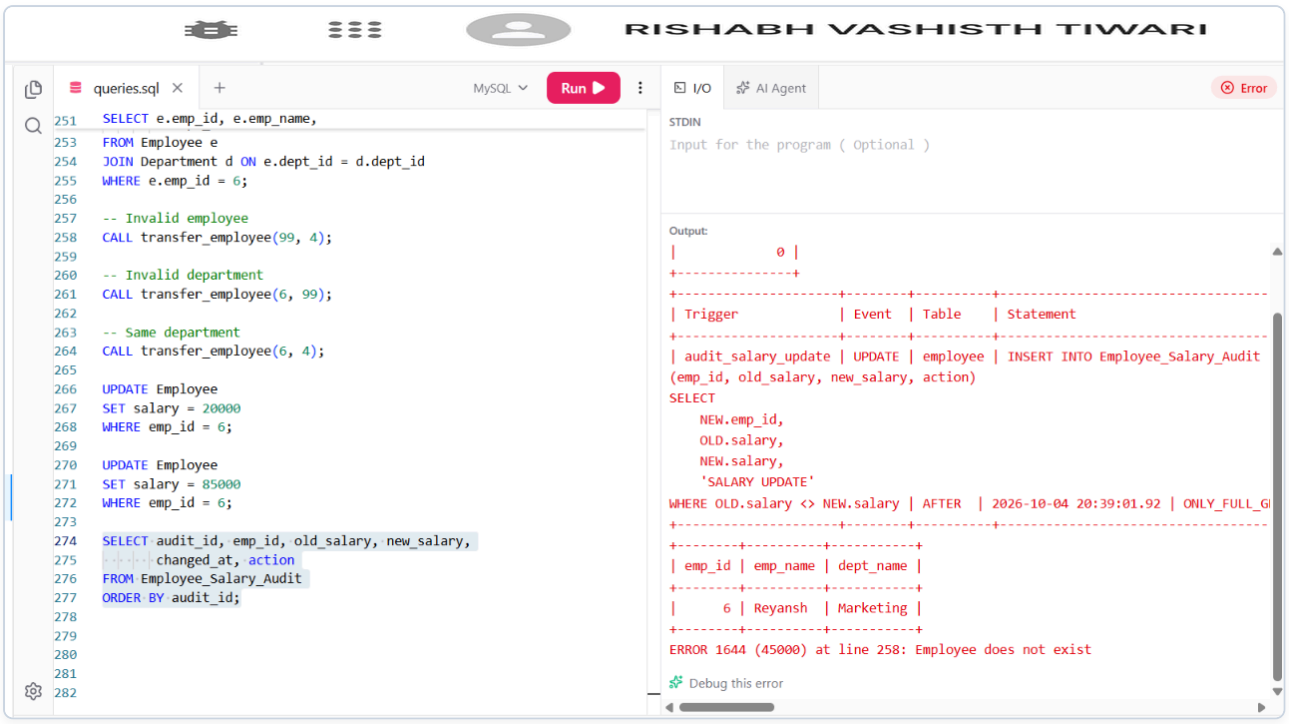
Test Salary Validation Output

9. TEST SALARY AUDIT LOG

Display the audit entries created by successful salary changes.

```
SELECT audit_id, emp_id, old_salary, new_salary,
changed_at, action
FROM Employee_Salary_Audit
ORDER BY audit_id;
```

Output




```
251 SELECT e.emp_id, e.emp_name,
253 FROM Employee e
254 JOIN Department d ON e.dept_id = d.dept_id
255 WHERE e.emp_id = 6;
256
257 -- Invalid employee
258 CALL transfer_employee(99, 4);
259
260 -- Invalid department
261 CALL transfer_employee(6, 99);
262
263 -- Same department
264 CALL transfer_employee(6, 4);
265
266 UPDATE Employee
267 SET salary = 20000
268 WHERE emp_id = 6;
269
270 UPDATE Employee
271 SET salary = 85000
272 WHERE emp_id = 6;
273
274 SELECT audit_id, emp_id, old_salary, new_salary,
275     changed_at, action
276 FROM Employee_Salary_Audit
277 ORDER BY audit_id;
278
279
280
281
282
```

```
|            0 |
+---------------+
+--------------------+--------+----------+--------------------------------
| Trigger            | Event  | Table    | Statement
+--------------------+--------+----------+--------------------------------
| audit_salary_update | UPDATE | employee | INSERT INTO Employee_Salary_Audit
(emp_id, old_salary, new_salary, action)
SELECT
    NEW.emp_id,
    OLD.salary,
    NEW.salary,
    'SALARY UPDATE'
WHERE OLD.salary <> NEW.salary | AFTER  | 2026-10-04 20:39:01.92 | ONLY_FULL_GI
+--------------------+--------+----------+--------------------------------
+--------+----------+-----------+
| emp_id | emp_name | dept_name |
+--------+----------+-----------+
|      6 | Reyansh  | Marketing |
+--------+----------+-----------+
ERROR 1644 (45000) at line 258: Employee does not exist
```

Test Salary Audit Log Output

RESULT

The transfer_employee procedure was created with validation, transaction control and error handling. Salary validation and audit triggers were implemented and edge cases were tested.

CONCLUSION

Stored procedures centralize controlled database operations, while triggers automatically enforce validation rules and maintain an audit trail for important changes.